

Autonomous Networking

Networks that observe, decide, and adapt

Gaia Maselli

Department of Computer Science
Sapienza University of Rome

A.Y. 2026 – 2027

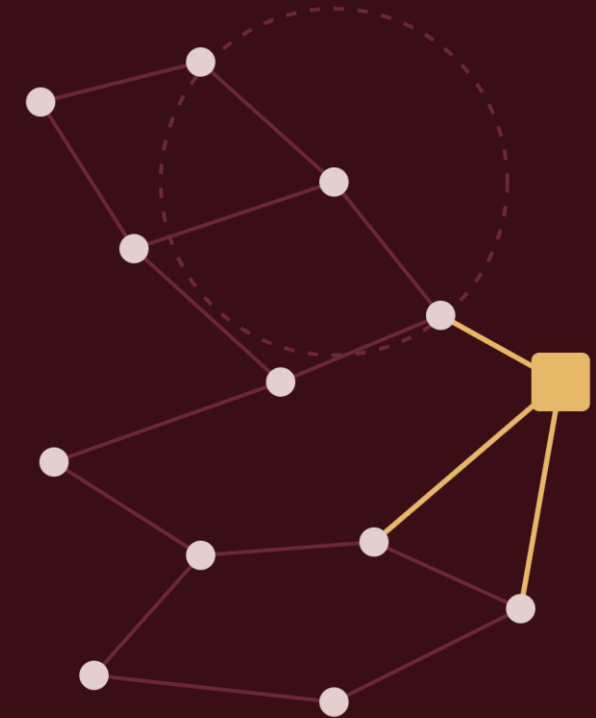


SAPIENZA
UNIVERSITÀ DI ROMA



Reinforcement Learning

Why networks need to learn, and the basic notions of RL



Today's plan

01	Autonomous networks	Where RL fits in the course
02	A case study	A battery-free smart home that needs a protocol that learns
03	Why reinforcement learning	Learning by interaction, and what makes RL different
04	Rewards and exploration	Long-term goals, and the exploration–exploitation dilemma
05	Agent and environment	Observation, history and state
06	Inside an agent	Policy, value function and model
07	Back to the network	The smart home as an RL problem

The course in three parts

PART 1

Types of networks

- RFID backscatter networks
- Wireless sensor networks
- Drone networks
- IoT networks
- Robophysical networks

PART 2 • TODAY

RL basics

- Agent, environment, reward
- Exploration vs exploitation
- State, policy, value function
- Bandits and Q-learning

PART 3

RL-based protocols

- MAC: who talks, and when
- Routing: who forwards, and where
- Learned from the network's own feedback

The first two parts run in parallel, not one after the other.

What is an autonomous network?

A network that works with no human intervention, like an independent organism.

AN AUTONOMOUS NETWORK

It involves intelligence

It learns from the environment, and reacts accordingly

Autonomy at the networking level

AT THE NETWORKING LEVEL

Communication (MAC)

- Who talks, and when
- Sharing a channel among many nodes

AT THE NETWORKING LEVEL

Routing

- Who forwards, and where
- Finding a path through the network

We do not address autonomy of robots or vehicles: **we make communication and routing autonomous.**

RL enters in parallel with the networks

LECTURES SO FAR AND NEXT

Introduction

RFID MAC

Sensor networks

RL basics

Bandits

Q-learning

already done / tomorrow

RL tools, introduced in parallel

Every new network brings a new problem to learn; **every RL tool brings a new way to solve it.**

FROM RFID TO RL

A battery-free smart home

Video camera

Presence sensor

Smoke sensor

Light sensor

Thermostat

Smart meter

TV remote

Light switch

Game controller

All of them work **without batteries**: they live on the energy they harvest.

Each device has its own pace

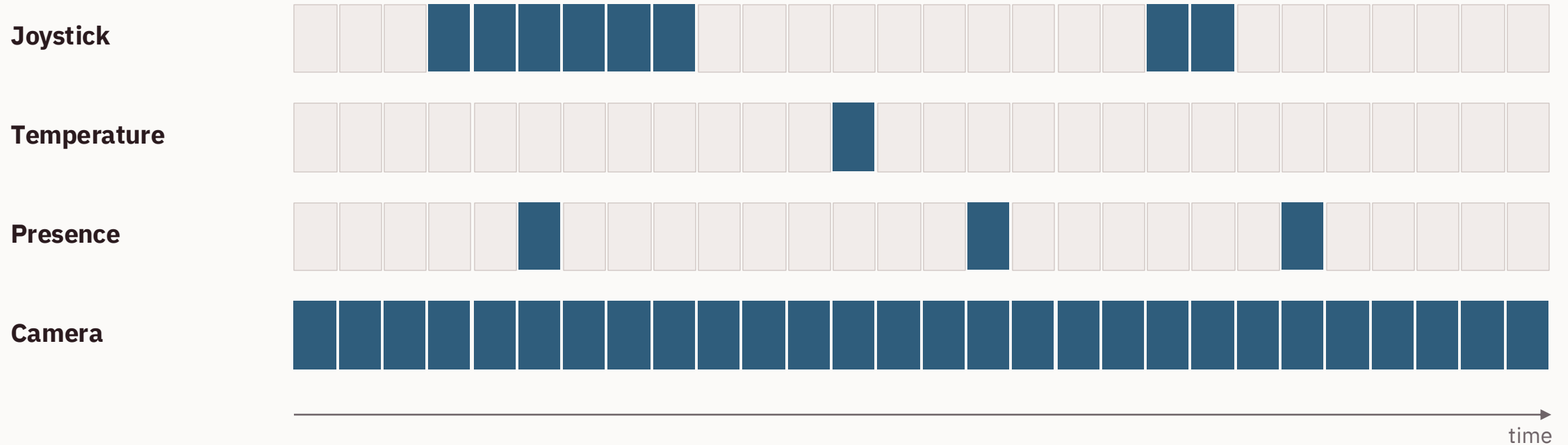
Device	When it has new data
Joystick	Only when it is used: bursts
Temperature sensor	Depends on the state: normal, fire, ...
Presence sensor	Only when someone walks by
Video camera	Continuously

Each device has its own pace, and the pace depends on the environment and on the user.

FROM RFID TO RL

Data is generated spontaneously

NEW DATA, SLOT BY SLOT (BLUE = A NEW SAMPLE IS READY)

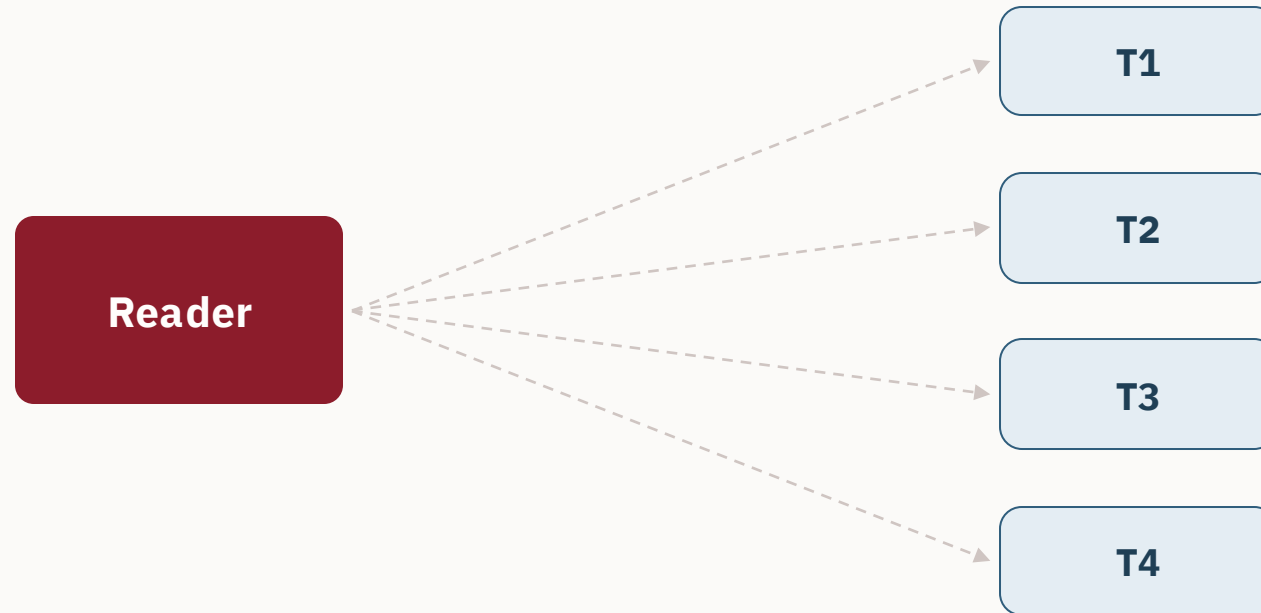


The ideal behaviour

A device should transmit only when it has new data.

Anything else wastes time, energy and bandwidth.

The catch: tags cannot speak first



THE CATCH

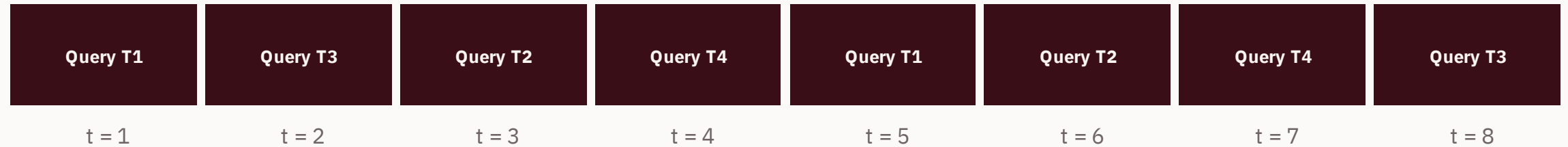
Backscatter devices cannot speak first

- They have no battery and no transmitter of their own
- They can only reflect the reader's signal, when the reader queries them

The reader decides who is allowed to talk.

The reader queries one device per slot

ONE QUERY PER TIME STEP



- Time is divided into pre-fixed slots
- In each slot the reader can query exactly one device

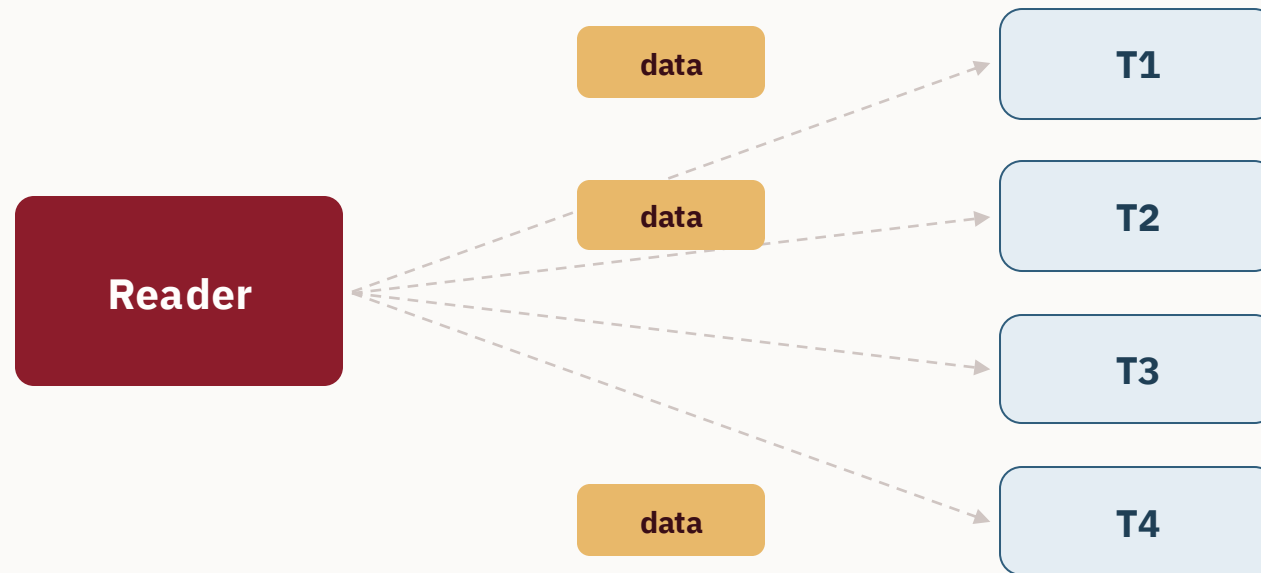
THE QUESTION

In which order should the reader query the tags?

Let us look at the obvious answers first.

Option 1: random access

ALL THE TAGS ANSWER IN THE SAME SLOT



OPTION 1

Random access: collisions

- Tags with new data answer in a random slot
- If two answer together, the reader receives nothing

The more tags have data, the more collisions: the channel is wasted.

Option 2: fixed round-robin

OPTION 2: FIXED ASSIGNMENT, ROUND-ROBIN

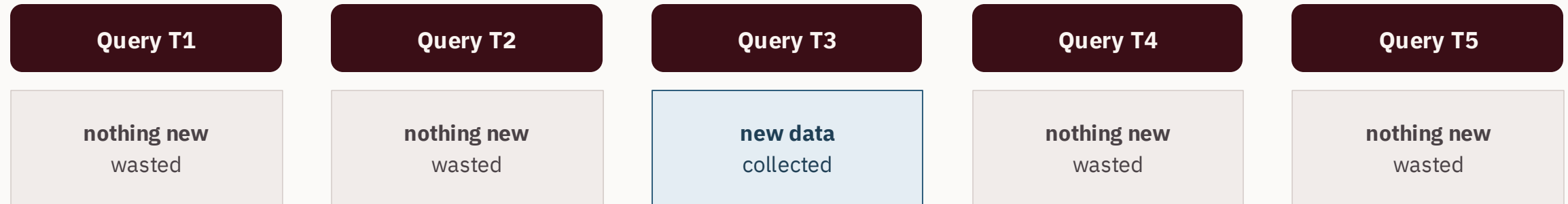


- Every tag is visited in turn, in a pre-fixed order
- No collisions: in each slot only one tag is queried

Collisions are gone. But the order knows nothing about who has data.

Problem 1: wasted queries

WASTED QUERIES

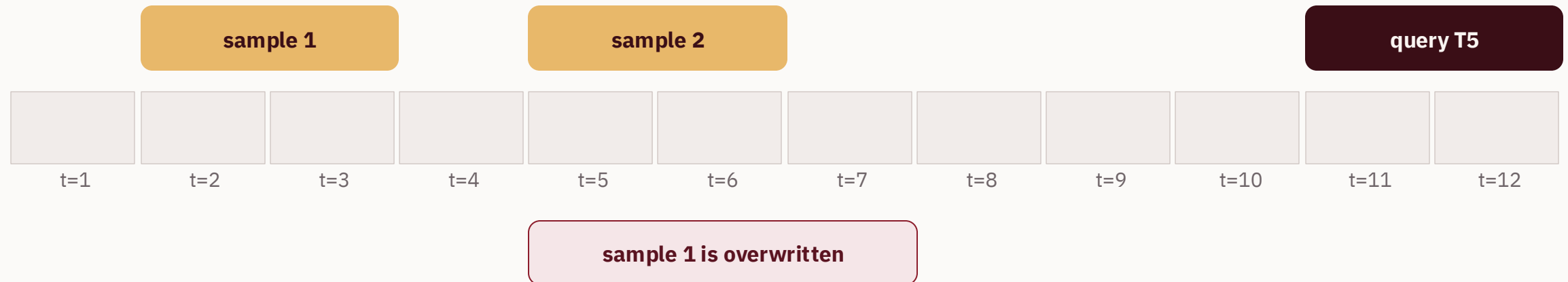


- We ask a tag that has nothing new since the last time

Here 4 queries out of 5 are wasted: time slots that could have served someone else.

Problem 2: loss of data

TAG T5 PRODUCES DATA FASTER THAN THE READER VISITS IT

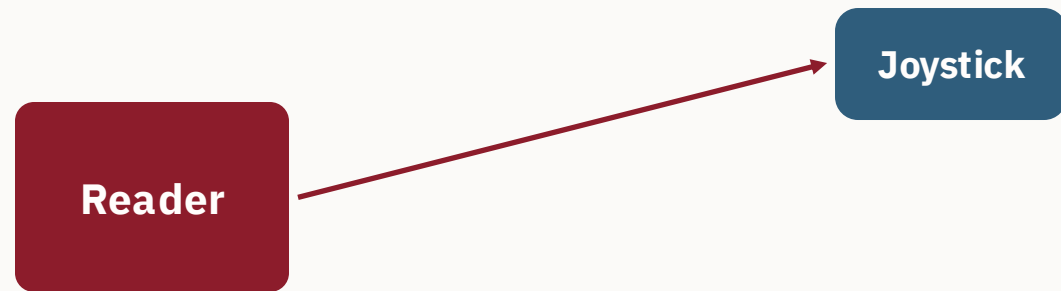


- Sample 2 arrives before sample 1 was read: the reader will only ever see sample 2

Loss of data: a new sample arrives before the old one is read.

Problem 3: redundant queries

THE OPPOSITE MISTAKE



- A device is queried many times for the same sample
- The first query reads it, the others get nothing new

Redundant queries are the flip side of a rigid policy: we over-visit slow devices and under-visit fast ones.

What we want

GOAL

A MAC protocol that adapts to the environment by itself: **query a device only when it has new data, and lose none.**

No fixed schedule can do that: it depends on what happens in the house, and it keeps changing.

A PROTOCOL THAT LEARNS FROM THE ENVIRONMENT

Nobody can write the right order in advance.

The answer is reinforcement learning

Learn by interacting with the environment, from its feedback.

The foundational question

How can an intelligent agent learn to make a good sequence of decisions?

The foundational question of reinforcement learning.

Piece 1: learn

- The agent does not know in advance how its decisions affect the world
- It does not know which decisions lead to good outcomes
- It has to acquire this information through experience

PIECE 1

Learn

Find out by doing, not by being told

Piece 2: a sequence of decisions

- Not just one decision, but a whole sequence
- Each decision changes what the agent will face next
- So a decision must be judged by where it leads

PIECE 2

A sequence of decisions

Which tag to query now changes what we know at the next slot

Piece 3: good

- We need a measure of utility for the decisions that are made
- In RL this measure is a numerical reward signal

PIECE 3

Good

“Good” means: it gets a high reward over time

Reinforcement learning in one line

IN ONE LINE

RL is learning what to do: how to map situations to actions so as to **maximize a numerical reward signal.**

Learning by interaction

- An agent can figure out how the world works by trying things and seeing what happens
- This is what we think people and animals do

AN EXAMPLE

An infant waving its arms

- No explicit teacher
- But a sensorimotor connection to its environment
- A wealth of information about cause and effect, and about how to achieve goals

Learning online

- We do not compute a good behaviour once and then execute it open-loop
- We keep learning while interacting with an ever-changing world
- We expect agents to get things wrong and to refine their understanding as they go

THE SECOND IDEA

Learning online

- Not only from a fixed dataset
- While acting, with the most recent experience

Why online: the world is not static

- The weather changes
- We meet new situations
- Our goals change

An agent that integrates its most recent experience does better **than one that tries to memorize how the world works.**

Characteristic 1: trial-and-error search

The learner is not told which actions to take.

It must discover which ones yield the most reward by trying them.

Characteristic 2: delayed reward

- An action affects the immediate reward
- It also affects the next situation
- And, through it, all the subsequent rewards

CHARACTERISTIC 2

Delayed reward

What you do now may pay off only much later

Sensation, action and goal

- | | | |
|---|------------------|--|
| 1 | Sensation | sense the state of the environment, to some extent |
| 2 | Action | take actions that affect that state |
| 3 | Goal | have a goal, or goals, related to the state of the environment |

Any method well suited to solving such problems is, for us, a reinforcement learning method.

The reward signal

- A reward R_t is a **scalar feedback signal**
- It indicates how well the agent is doing at step t

THE REWARD

A single number

- Positive: good
- Negative: bad
- Zero: nothing happened

THE RL PROBLEM

Maximize the cumulative reward

The agent's job is to maximize the cumulative reward.

Not the reward of the next step: the sum over time.

THE REWARD HYPOTHESIS

All goals can be described by the maximization of expected cumulative reward.

THE RL PROBLEM

Reward example: a walking robot

EXAMPLE OF REWARD

Make a humanoid robot walk

Positive reward

+ forward motion

Negative reward

– falling over

THE RL PROBLEM

Reward example: battery-free networks

EXAMPLE OF REWARD

Communication in battery-free environments

Positive reward

+ querying a device with new data

Negative reward

– querying a device with nothing new

Reward design in the smart home

Event	Reward (illustrative)
Query returns a new sample	+1
Query returns nothing new	-1
A sample is overwritten before it is read	-5

The numbers are an illustration: designing the reward is part of designing the protocol.

Long-term consequences

- Actions may have long-term consequences
- Reward may be delayed
- It may be better to sacrifice immediate reward to gain more long-term reward

SEQUENTIAL DECISION MAKING

Select actions to maximize total future reward

Not the next reward, the whole future

Exploitation

- To obtain a lot of reward, an agent must prefer actions that were tried in the past and found effective
- Take advantage of the best option we know

THE FIRST SIDE

Exploit

Use what you know

Exploration

- To discover effective actions, the agent has to try actions it has not selected before
- Find more information about the environment, taking some risk

THE OTHER SIDE

Explore

Try something new

Where do we eat?

Your favourite restaurant is around the corner

Go there every day: you are confident of what you get, but you miss the chance of finding a better one

Try new places all the time

Very likely you will eat unpleasant food from time to time

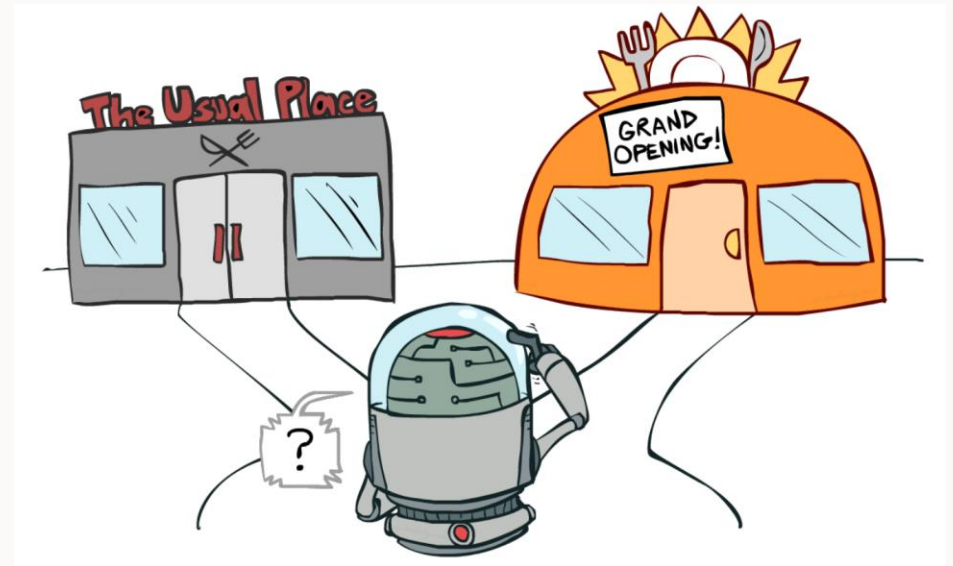


Image source: UC Berkeley AI course

The dilemma: incomplete information

The dilemma comes from incomplete information.

We must gather enough of it to decide well, while keeping the risk under control.

Neither is enough

- Neither can be pursued exclusively without failing at the task
- Try a variety of actions, and progressively favour those that appear to be best

Explore only: we never collect the reward we know. **Exploit only:** we may be stuck with a bad option.

Stochastic tasks need many trials

- On a stochastic task, each action must be tried many times
- Only then do we get a reliable estimate of its expected reward

STOCHASTIC TASKS

Why many trials?

- One sample can be lucky or unlucky
- The estimate improves with the number of samples

Short-term sacrifices may pay off

- One exploration trial may be a total failure
- But it warns us not to take that action too often in the future
- The best long-term strategy may involve short-term sacrifices

THE PRICE OF EXPLORING

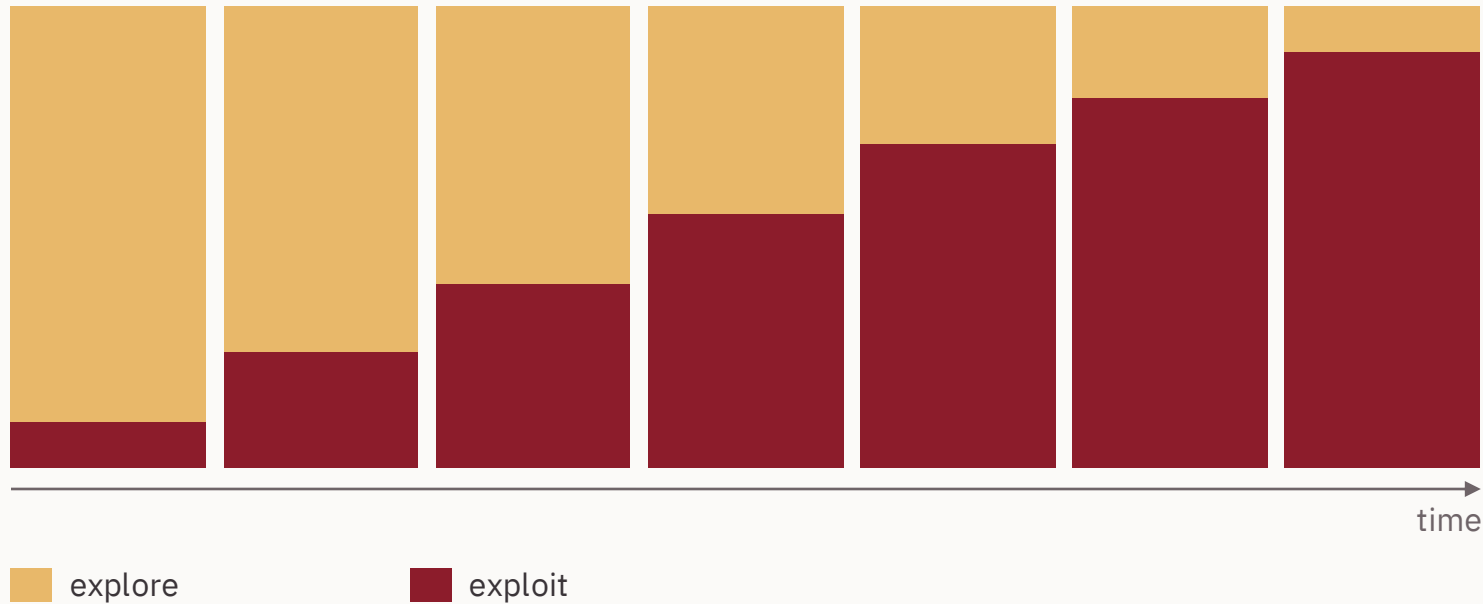
Failures are information

A bad trial is not wasted

EXPLORATION VS EXPLOITATION

A typical recipe

A TYPICAL RECIPE (ILLUSTRATION)



Explore early, exploit later

- At the start we know nothing
- As estimates become reliable, we exploit more

Explore or exploit in the smart home

IN THE SMART HOME

Exploit

Keep querying the tags that were rich in data

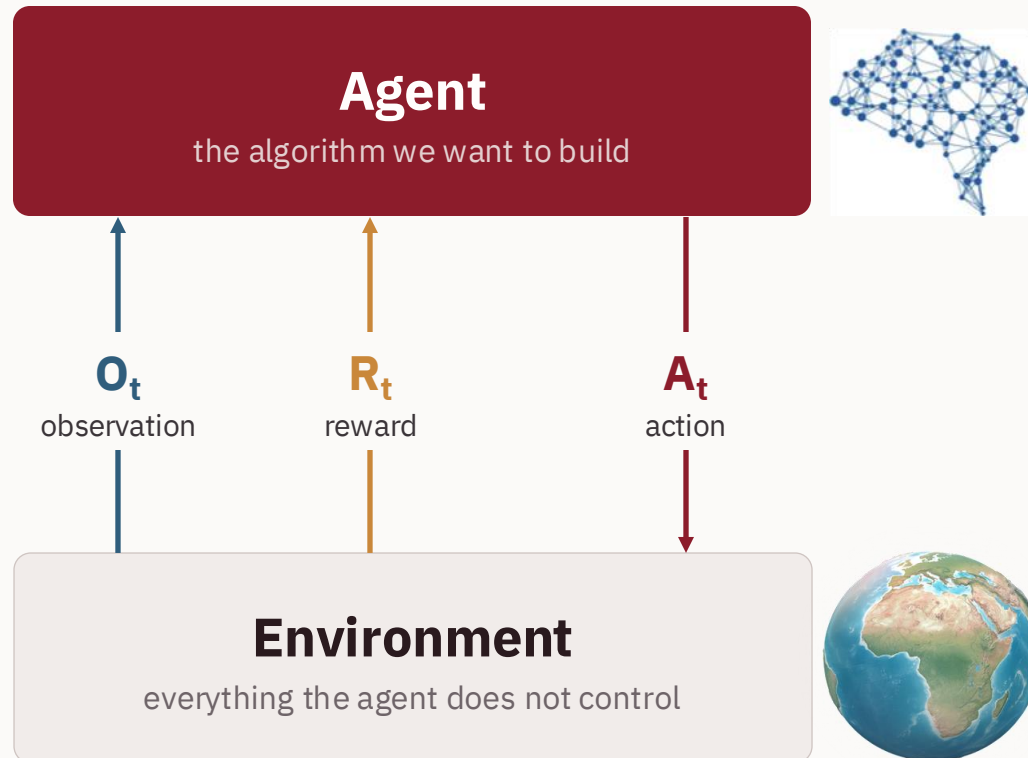
IN THE SMART HOME

Explore

Query a tag we have not seen for a while: it may have new data we know nothing about

Next, we study this dilemma in its simplest form: **multi-armed bandits**.

Agent and environment



THE FRAMEWORK

Two actors

- The agent: we want to build it
- The environment: we only influence it through actions

What the agent does at each step



THE AGENT

At each step t the agent

- executes action A_t
- receives observation O_t
- receives scalar reward R_t

What the environment does at each step



THE ENVIRONMENT

At each step t the environment

- receives action A_t
- emits observation O_t
- emits scalar reward R_t
- then t increments

The framework in the smart home

Framework	Smart home
Agent	The reader
Environment	The house: tags, people, devices
Action A_t	Query tag i in this slot
Observation O_t	The tag's answer: new data, or nothing
Reward R_t	Positive if the data was new, negative if not

The history

THE SEQUENCE OF ACTIONS, OBSERVATIONS AND REWARDS UP TO TIME t



$$H_t = A_1, O_1, R_1, \dots, A_t, O_t, R_t$$

History: all the observable variables up to time t , like the sensorimotor stream of a robot.

What happens next depends on the history

- The agent selects actions
- The environment selects observations and rewards
- So what happens next depends on the history

WHY THE HISTORY MATTERS

$H_t \rightarrow$ what happens next

The history contains everything that could be used to predict the future

The state

- The state is the information used to determine what happens next
- Formally, it is a function of the history

STATE

$$\mathbf{S}_t = \mathbf{f}(\mathbf{H}_t)$$

E.g. the last observation, or the last four observations

Why we need a state

The history grows forever.

The state is a compact summary of it, enough to decide.

Environment state

- The environment's private representation
- Whatever data the environment uses to pick the next observation and reward
- Usually not visible to the agent



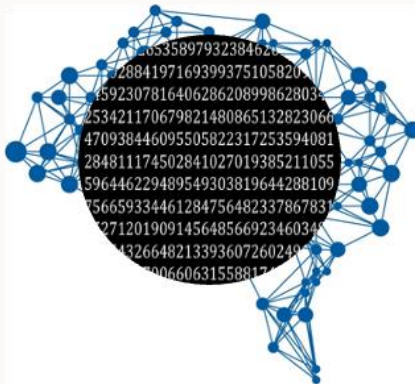
ON THE ENVIRONMENT SIDE

Environment state S^e_t

In the smart home: which tags hold new data right now

Agent state

- The agent's internal representation
- Whatever information the agent uses to pick the next action
- The information used by RL algorithms
- It can be any function of the history



ON THE AGENT SIDE

Agent state $S^a_t = f(H_t)$

In the smart home: what the reader remembers about its past queries

Fully observable environments

Agent state = environment state

The agent directly observes the environment state.

Fully observable: what the agent sees is all there is to know.

Partially observable environments

- The agent observes the environment only indirectly
- A robot with camera vision is not told its absolute location
- A trading agent sees only the current prices

THE COMMON CASE

Partially observable

Agent state $\neq$ environment state

In the smart home: the reader does not see which tags have data; it only sees the answers it gets.

A GENERAL RL FRAMEWORK

Example: what will the rat get?

A RAT OBSERVES THREE EPISODES: WHAT SHOULD EPISODE 3 END WITH?

1	Light	Light	Lever	Bell	→	Shock
2	Bell	Light	Lever	Lever	→	Food
3	Lever	Light	Lever	Bell	→	?



What we believe will happen next depends on how we represent the state.

A GENERAL RL FRAMEWORK

Rat: the last three items

AGENT STATE = THE LAST THREE ITEMS



Episodes 1 and 3 both end with Light, Lever, Bell: **prediction = Shock.**

A GENERAL RL FRAMEWORK

Rat: the counts

AGENT STATE = THE COUNTS OF LIGHTS, BELLS AND LEVERS



Episode 3 has the same counts as episode 2 (1 light, 1 bell, 2 levers): **prediction = Food.**

A GENERAL RL FRAMEWORK

Rat: the complete sequence

AGENT STATE = THE COMPLETE SEQUENCE



We have never seen this sequence: **no prediction is possible.**

Ways to build the agent state

THE AGENT MUST CONSTRUCT ITS OWN STATE: THREE OPTIONS

OPTION 1

Complete history

Keep everything that was observed

Nothing is lost, but it keeps growing

OPTION 2

Beliefs about the environment state

Assign a probability to each possible state

Handles partial observability

OPTION 3

A function of the history

E.g. a linear combination of the current and the previous state

Compact, by design

The environment does not hand the state over: the agent has to build it.

A candidate state for the smart home

- For each tag: what happened the last times we queried it
- And how long ago it was queried

IN THE SMART HOME

A natural candidate state

Per tag: time since the last query, outcome of the last query

Not the true state of the house, **but enough to decide who to query next.**

Inside an RL agent: three components

An RL agent may include one or more of these components.

Policy

What to do

Value function

What is good in the long run

Model

What follows what

Policy: what to do

- A policy defines the agent's behavior at a given time
- It is a map from state to action
- It is the core of an RL agent: it alone is sufficient to determine behavior

COMPONENT 1

Policy

What to do

Deterministic and stochastic policies

TWO KINDS OF POLICY

Deterministic

$$\pi(s) = a$$

A simple function of the state

TWO KINDS OF POLICY

Stochastic

$$\pi(a | s) = P[A_t = a | S_t = s]$$

A probability for each action

Example: “always query the tag that waited longest” is deterministic; “query the tag that waited longest with probability 0.9, another tag otherwise” is stochastic.

The reward changes the policy

- The reward signal is the primary basis for altering the policy
- If an action chosen by the policy is followed by low reward, the policy may change
- In that situation, it may select another action in the future

LEARNING

Reward changes the policy

This is how the agent learns

Value function: good in the long run

- The reward signal says what is good in an immediate sense
- The value function says what is good in the long run
- Without rewards there could be no values

COMPONENT 2

Value function

What is good in the long run

Value is a prediction of future reward

- The value of a state is a prediction of future reward
- The total reward the agent can expect to accumulate, starting from that state
- It is used to evaluate how good or bad states are

VALUE

A prediction of the future

How much reward do I get, going into the future from here?

The return and the discount factor

- The return is the sum of future rewards
- Each one is discounted by γ , between 0 and 1
- Near rewards count more than distant ones

THE RETURN

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

The value function

- The value of a state is the expected return
- Starting from that state, following the policy π

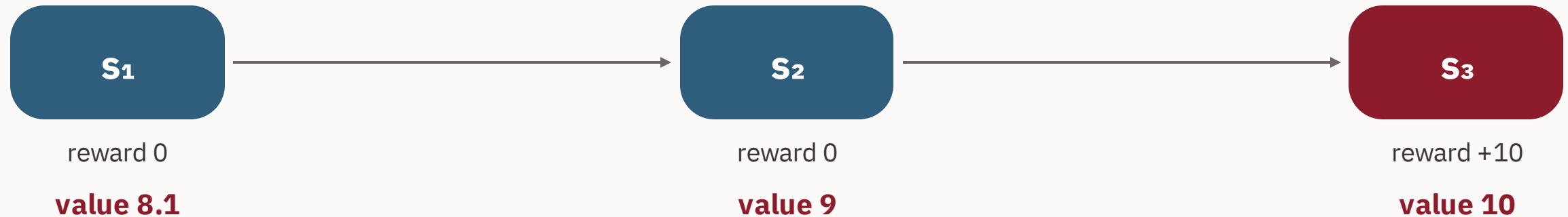
THE VALUE FUNCTION

$$v_{\pi}(s) = E_{\pi} [G_t \mid S_t = s]$$

Expected: the environment may be stochastic, so we average over what can happen.

Value example: low reward, high value

WITH $\gamma = 0.9$



From s₁ the agent receives 0 now, but it is two steps from +10: $0.9 \times 0.9 \times 10 = 8.1$. **Low reward now, high value.**

The components in the smart home

Component	In the smart home
Policy	Which tag to query next
Value function	How much new data we expect from querying each tag
Model	How the home produces data over time

We will learn the policy directly, and maybe a value function: **no model.**

Back to the smart home

- n heterogeneous battery-free devices
- Device i produces new data at its own Rate_i , which depends on the environment
- Devices must be queried by the reader
- One query per time step, in pre-fixed time slots

THE SMART HOME

Recap: the setting

The reader decides who talks

Formulate it as an RL problem

Agent ?

Who learns and decides?

Actions ?

What can it do at each time step?

Rewards ?

What should it be rewarded for?

State ?

What does it know about the tags?

FOR DISCUSSION

Formulate the problem as an RL problem.

REFERENCES

Further reading

R. S. Sutton, A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed., MIT Press, 2018 (free online). Chapter 1 for today; chapter 2 for bandits.

D. Silver, *UCL Course on RL*, Lecture 1: Introduction to Reinforcement Learning. The agent/environment/state framework, the rat example follow this lecture.

Restaurant illustration: UC Berkeley AI course.